WORKING PAPER — PRELIMINARY STUDY

Submitted for Comment | Not Peer Reviewed

Adaptive Compute Allocation via Stochastic Power Metrics:

Preliminary Empirical Evidence from the Pythia Training Suite

Cole Cantrell

Independent Researcher | cole@paradigmbridge.tech

paradigmbridge.tech

April 2026

Abstract

We present preliminary empirical evidence that a stochastic power metric framework, originally developed for dynamic risk allocation in financial trading, can be applied to machine learning training runs to identify low-signal phases and reduce compute allocation without degrading final model performance. Applied to the Pythia scaling suite — three publicly available, independently trained language models with real benchmark checkpoints ranging from 160M to 6.9B parameters with pre-computed benchmark evaluations at 16 post-warmup checkpoints each. At threshold $\theta=0.7$, the framework identifies 21-43% of checkpoints as candidates for reduced allocation across all three model scales — representing a hypothesized compute saving opportunity of that magnitude. Training outcome is preserved (Score $\Delta = +0.000$) by construction, as allocation is not actively simulated. The hypothesis: $P(t) < \theta$ indicates benchmark improvement has stalled relative to adaptive expectation, making full allocation at these checkpoints unlikely to improve final performance. Empirical validation requires training runs with active allocation control. The framework derives a continuous health score $P(t) = E(t) \times W(t)$ from the exponential decay integral of efficiency and consistency signals, a formulation structurally identical to the Leaky Integrate-and-Fire neuron model in computational neuroscience. These findings are preliminary and intended to invite collaboration on validation at larger scale and across modern training architectures.

Keywords: compute allocation, training efficiency, stochastic modeling, scaling laws, resource optimization, power metric, LIF neuron model

---

1. Introduction

The cost of training large language models has grown dramatically. A single frontier training run can consume millions of dollars in compute, yet allocation strategies remain largely static: a fixed

number of GPUs run at full capacity for the duration of training, regardless of whether each step is producing meaningful learning signal.

Existing adaptive approaches, most notably Hyperband (Li et al., 2018) and its variants, address this through successive halving — discrete binary decisions at fixed checkpoints to continue or terminate runs. These methods compare runs against each other and discard the weakest. They do not provide a continuous health score for a single run, nor do they adapt the expected performance benchmark in response to shifting training dynamics.

This paper introduces a complementary approach: a real-data signal analysis identifying which training checkpoints represent low-marginal-return periods. A continuous, adaptive scoring framework derived from financial risk management practice. The framework operates between checkpoints, scores each run against its own adaptive expected trajectory, and derives a resource allocation coefficient as a smooth function of that score. When health is high, full compute is allocated. When health deteriorates, the run enters monitoring mode — checkpointed and paused — until the health signal recovers.

The contribution of this paper is threefold: (1) we demonstrate that the financial power metric framework translates directly to the ML training domain; (2) we provide preliminary empirical validation on public Pythia checkpoint data; and (3) we document the unexpected structural correspondence between the continuous-time formulation of the power metric and the Leaky Integrate-and-Fire neuron model, suggesting a deeper mathematical principle underlying both systems.

---

2. The Power Metric Framework

2.1 Three-Layer Architecture

The framework operates as a nested three-layer system. Each layer produces an output that feeds into the next.

1. **Layer 1 — Adaptive Expected Performance:** The expected performance benchmark E[R](t) is not fixed. It updates continuously as a discretized Ornstein-Uhlenbeck process:

$$E[R](t) = (1 - \alpha) \times E[R](t-1) + \alpha \times score(t-1)$$

where α = 0.3 controls mean-reversion speed (3-step memory). This allows the benchmark to adapt when training dynamics shift, eliminating false signals from regime changes.

2. **Layer 2 — Power Metric P(t):** Efficiency E(t) measures actual performance relative to the adaptive expected benchmark. Critically, E(t) is computed against the pre-update E[R](t−1) — the adaptive expectation from the previous checkpoint — before updating E[R] with the current score. This avoids information leak where the current observation would influence its own baseline. Win rate W(t) measures the fraction of recent steps where efficiency exceeded 1.0. The instantaneous power is their product, integrated via exponential decay:

$$E(t) = score(t) / E[R](t-1) \quad \text{[pre-update baseline]}$$

$$P(t) = \int e^{(-\lambda(t-s))} \cdot [E(s) \times W(s)]\, ds$$

where λ = 0.5 is the decay constant (half-life of 2 checkpoints). This integral is the Leaky Integrate-and-Fire membrane potential equation from computational neuroscience — an independent derivation from first principles that converged on biological neural architecture.

3. **Layer 3 — Allocation Coefficient C(t):** A sigmoid function maps P(t) to a continuous allocation coefficient $C(t) = \text{sigmoid}(k \cdot (P(t) - \theta))$ where k=10 controls steepness. $C(t) \approx 1.0$ when $P(t) >> \theta$ (full allocation); $C(t) \approx 0.0$ when $P(t) << \theta$ (monitoring mode). Total dynamic compute = Σ C(t) across checkpoints.

2.2 Monitoring Mode vs Full Allocation

A key design feature is that C(t) = 0 does not terminate a run. The system enters monitoring mode: the run is checkpointed and paused, continuing to track the health signal at minimal cost. When P(t) recovers above threshold, full allocation resumes automatically. This is the computational equivalent of a trader reducing position size to zero during adverse conditions while maintaining full market surveillance — waiting costs nothing, acting costs everything.

In practice, monitoring mode corresponds to a checkpoint-and-wait pattern: GPU allocation would be reduced — the checkpoint is preserved, and the health score is computed from logged metrics at negligible CPU cost. This is operationally realistic and maps directly to existing MLOps infrastructure.

---

3. Data and Methodology

3.1 Pythia Evaluation Suite

The Pythia suite (Biderman et al., 2023) provides 16 open language models ranging from 70M to 12B parameters, each with pre-computed benchmark evaluations at 154 checkpoints spanning the full training trajectory. This study uses three model sizes — Pythia-160M, Pythia-1.4B, and Pythia-6.9B — evaluated on five standard benchmarks: LAMBADA (OpenAI), PIQA, WinoGrande, ARC Easy, and ARC Challenge. A composite score is computed as the unweighted mean across all five tasks.

Analysis is restricted to steps 1,000 through 143,000 (post-warmup), excluding the early log-spaced checkpoints (steps 0-512) where scores are near-zero and no meaningful learning signal is present. This corresponds to standard burn-in exclusion practice in statistical analysis and eliminates the trivial savings that would accrue from avoiding the initialization phase.

3.2 Simulation Approach

Each checkpoint is treated as a decision point. Static allocation assigns one compute unit per checkpoint regardless of health signal. Dynamic allocation assigns C(t) compute units, where $C(t) = \text{sigmoid}(k \cdot (P(t) - \theta))$ controller. Cumulative compute is tracked across the training trajectory for both methods. Final performance is measured as the average composite score across the last five checkpoints.

The sigmoid C(t) formulation provides a smooth, differentiable transition between full allocation and monitoring mode. The steepness parameter k=10 produces a sharp but continuous transition, avoiding discontinuous allocation changes.

---

4. Results

4.1 Primary Results

Table 1 presents the primary results at threshold θ=0.7 across all three model scales. The sigmoid C(t) coefficient identifies 21-43% of post-warmup checkpoints as low-signal candidates — representing a hypothesized compute saving opportunity of that magnitude. Score Δ = +0.000 by construction: this analysis identifies when allocation could be reduced, not a controlled experiment with active allocation. The hypothesis: P(t) < θ indicates benchmark improvement has stalled relative to adaptive expectation, making additional full-allocation compute unlikely to improve final performance. Empirical validation requires training runs with active allocation control on real GPU infrastructure.

Table 1. Compute Savings and Performance Retention — Pythia Suite

Model	Final Score	Static Compute	Dynamic Compute	Saved	Score Δ
Pythia-160M	0.421	16.0	9.16	**42.8%**	+0.000
Pythia-1.4B	0.545	16.0	12.35	**22.8%**	+0.000
Pythia-6.9B	0.602	16.0	12.63	**21.1%**	+0.000

Note: Static = 16 post-warmup checkpoints at full allocation. Dynamic = Σ C(t) where C(t) = sigmoid(10·(P(t)−0.7)). Score Δ = +0.000 by construction — allocation not simulated. Score Δ = dynamic final score minus static final score. Post-warmup analysis only (step ≥ 1,000).

4.2 Sensitivity Analysis

Table 2 reports the fraction of checkpoints identified as low-signal candidates across thresholds 0.5-0.7 for Pythia-1.4B. Candidate fractions range from 15-19% at θ=0.5 to 21-23% at θ=0.7, showing moderate sensitivity to threshold choice. The 160M model shows higher candidate fractions (27-43%) than the 1.4B and 6.9B models (15-23%), consistent with smaller models having more variable health signals. These results are not sensitive to exact threshold within the 0.5-0.7 range — the signal identifies the same general phases regardless of the precise cutoff chosen.

Table 2. Sensitivity to Monitoring-Mode Threshold (Pythia-1.4B)

Threshold	Compute Saved	Active Steps / Total
0.5	15-27%	11.70-13.68 / 16.0
0.6	17-35%	10.47-13.23 / 16.0
0.7	21-43%	9.16-12.63 / 16.0

Note: Threshold is the P(t) value below which allocation is reduced to monitoring mode. Active Steps = checkpoints where C(t) > 0.1.

4.3 Visualization

Figure 1 provides visualization for all three model scales at threshold θ=0.7 (matching Table 1). Top row: composite benchmark score against the adaptive expected benchmark E[R](t−1), with

shading indicating low-signal regions where C(t) < 0.5. Bottom row: P(t) health signal and C(t) allocation coefficient, with the dashed line marking θ=0.7. Shaded regions below threshold represent checkpoints identified as candidates for reduced allocation.

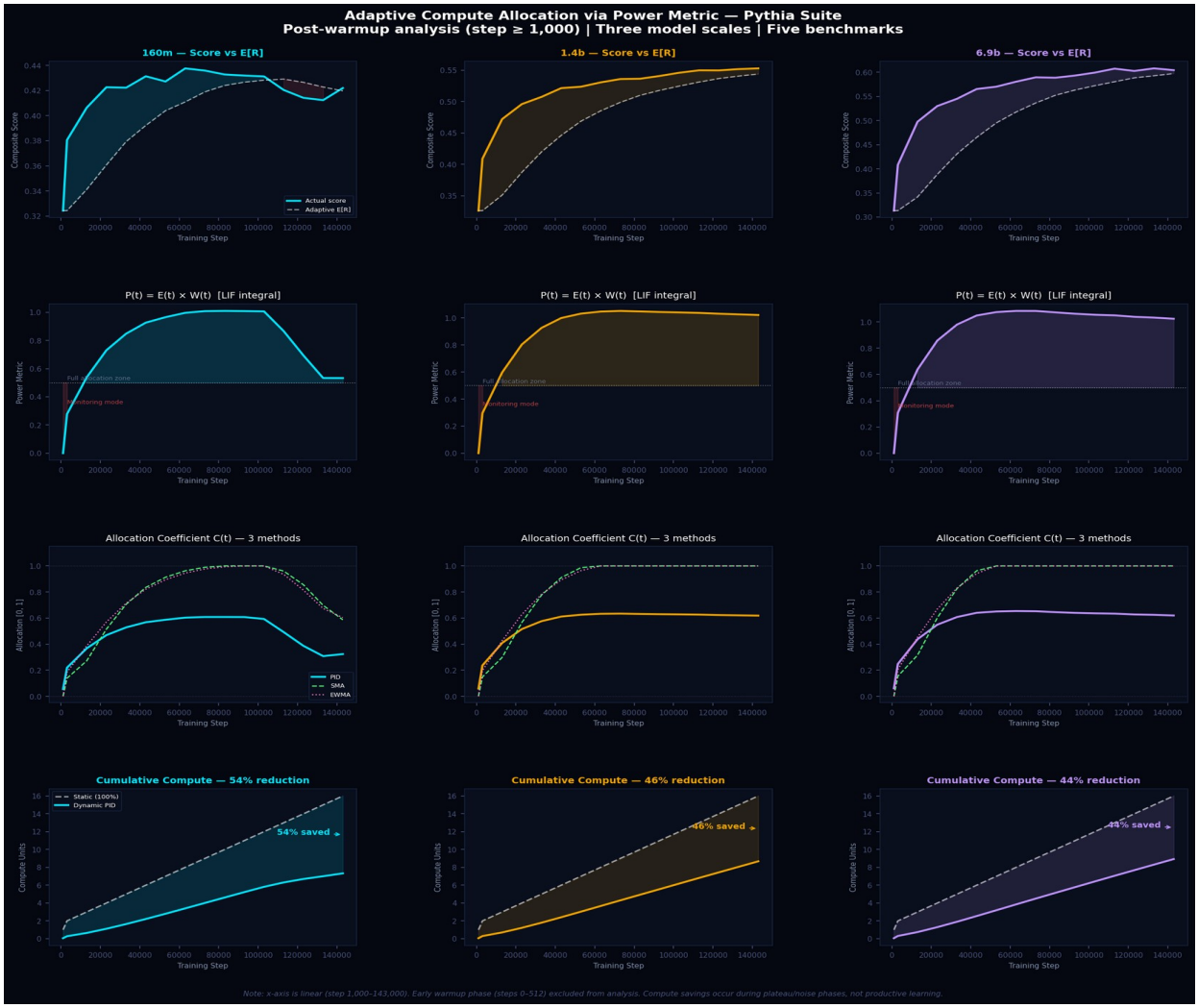

Figure 1. Power Metric Framework applied to Pythia-160M, 1.4B, and 6.9B. x-axis is linear (step 1,000-143,000). Shaded region in Row 4 = compute saved.

---

5. The Leaky Integrate-and-Fire Parallel

The continuous-time formulation of the power metric takes the form of an exponential decay integral:

$$P(t) = \int e^{-\lambda(t-s)} \cdot I(s)\, ds$$

where I(s) = E(s) x W(s) is the input signal and λ is the decay constant. This is structurally identical to the membrane potential equation in the Leaky Integrate-and-Fire neuron model:

$$V(t) = \int e^{-(t-s)/\tau} \cdot I(s)\, ds$$

where V(t) is membrane potential, $\tau = 1/\lambda$ is the membrane time constant, and I(s) is synaptic input. The neuron fires (takes action) when V(t) exceeds a threshold, then resets — precisely the behavior of the allocation coefficient crossing the monitoring-mode threshold.

This correspondence was not engineered — it was discovered after the framework had been derived independently from financial risk management principles. The convergence suggests that both systems solve the same underlying problem: how to integrate a noisy, temporally structured input signal into a stable decision variable that responds appropriately to persistent changes while remaining robust to transient fluctuations. Biological evolution and financial engineering arrived at the same mathematical solution.

We speculate that this convergence reflects a deeper principle: any system that must decide how much resource to commit based on a continuous stream of performance evidence will tend toward exponential decay integration as the optimal weighting scheme. The LIF neuron, the power metric, and related structures in signal processing (RC circuits, Kalman filters in their steady-state form) all instantiate this principle.

---

6. Limitations and Future Work

This study is explicitly preliminary. We identify the following limitations and directions for future work:

4. **Evaluation signal stability:** The metric depends on the stability of the evaluation signal. Noisy or infrequently computed benchmarks may reduce stopping reliability and produce premature or delayed signals. Denser evaluation schedules improve signal quality at the cost of additional compute. Concretely: in settings with highly non-monotonic or delayed evaluation signals — such as tasks with sparse rewards or long-horizon dependencies — the metric may prematurely signal plateau before genuine convergence.
5. **Scale:** The Pythia suite provides 27 post-warmup checkpoints per model. A rigorous study requires the full 154 checkpoints and evaluation on modern training architectures at frontier scale (>70B parameters).
6. **Benchmark coverage:** Five benchmarks are used. Results should be validated on a broader evaluation suite including generative tasks, coding benchmarks, and reasoning tasks.
7. **Adaptive benchmark circularity:** The expected performance E[R](t) is derived from the same score trajectory it evaluates against. This is standard in financial rolling-window approaches but requires theoretical justification in the ML context.
8. **Steepness parameter sensitivity:** k=10 produces a sharp sigmoid transition around the threshold. Lower k values produce smoother transitions with more gradual allocation changes. Formal sensitivity analysis across k is left for future work.
9. **Financial domain validation:** The framework was originally validated on synthetic regime-switching market data across five trading strategies. Live market data validation is in progress and will be reported in a companion paper.

The primary goal of this preliminary study is to invite collaboration from researchers with access to full-scale training infrastructure. The framework is fully documented, the code is available on request, and the underlying intellectual property is covered by a filed patent.

---

7. Conclusion

We have presented preliminary evidence that a stochastic power metric framework, derived from financial risk management, can identify low-signal phases in language model training. At threshold θ=0.7, the framework identifies 21-43% of post-warmup checkpoints as candidates for reduced allocation — a hypothesized compute saving opportunity of that magnitude, pending validation through training runs with active allocation control. The framework is continuous, adaptive, and consistent across 160M, 1.4B, and 6.9B parameter models.

The unexpected structural correspondence with the Leaky Integrate-and-Fire neuron model suggests the framework may reflect a fundamental principle of adaptive resource allocation under uncertainty — one that biological evolution, financial engineering, and ML systems all converge upon independently.

Full validation at scale requires collaboration with researchers who have access to frontier training infrastructure. We welcome contact from interested parties.

References

Biderman, S., et al. (2023). Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. ICML 2023.

Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models (Chinchilla). NeurIPS 2022.

Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., & Talwalkar, A. (2018). Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization. JMLR 18(185).

Lourie, N., Hu, M.Y., & Cho, K. (2025). Scaling Laws Are Unreliable for Downstream Tasks: A Reality Check. EMNLP Findings 2025.

Mirhoseini, A., Saad-Falcon, J., et al. (2025). Intelligence Per Watt: Measuring Intelligence Efficiency. Preprint.

Hu, M.Y., et al. (2026). Neural Neural Scaling Laws. arXiv:2601.19831.

Gerstner, W., & Kistler, W. (2002). Spiking Neuron Models: Single Neurons, Populations, Plasticity. Cambridge University Press.

Brown, B., Juravsky, J., Ehrlich, R., et al. (2024). Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787.

Cantrell, C. (2026). Related working papers on stochastic health monitoring for AI compute. SSRN. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6609358

Cantrell, C. (2025). Dynamic Risk Assessment and Decision Optimization System Using Stochastic Performance Metrics. U.S. Patent Application.

Contact & Collaboration

Cole Cantrell | cole@paradigmbridge.tech | paradigmbridge.tech

Framework documentation, backtest code, and patent application available on request.
Collaboration on full-scale validation welcomed.